
Semana 6 – Patrones Estructurales: Adapter, Decorator, Composite

OBJETIVO

Aplicar patrones estructurales en C++ y Python para mejorar la modularidad, integración de componentes y flexibilidad de software.

MARCO TEÓRICO

- **Adapter**: convierte una interfaz existente en otra compatible sin modificarla.
- **Decorator**: extiende funcionalidades en tiempo de ejecución.
- **Composite**: maneja jerarquías de objetos (ej. menús en interfaces gráficas).

DESARROLLO

1. Construir un **adaptador de pagos** que convierta dólares a soles.
2. Aplicar un **decorator** para añadir validación extra a funciones matemáticas.
3. Usar **Composite** para crear una estructura jerárquica de carpetas y archivos.

Adapter

También llamado: Adaptador, Envoltorio, Wrapper

Adapter es un patrón de diseño estructural que permite la colaboración entre objetos con interfaces incompatibles.

Construir un adaptador de pagos que convierta dólares a soles.

IMPLEMENTACIÓN PYTHON

```
# Clase original (Adaptee)
class PagoDolares:
    def pagar_dolares(self, monto):
        return "Pago realizado: " + str(monto) + " dolares"

# Adaptador (Adapter)
class AdaptadorPago:
    def __init__(self, pago):
        self.pago = pago

    def convertir_a_soles(self, monto_dolares):
        tipo_cambio = 3.70

        # Convertir dólares a soles
        monto_soles = monto_dolares * tipo_cambio

        return monto_soles
```

```
# Programa principal
pago = PagoDolares()

adaptador = AdaptadorPago(pago)

# Varias pruebas
dolar1 = 10
soles1 = adaptador.convertir_a_soles(dolar1)

dolar2 = 25
soles2 = adaptador.convertir_a_soles(dolar2)

dolar3 = 100
soles3 = adaptador.convertir_a_soles(dolar3)

print(pago.pagar_dolares(dolar1))
print("Equivale a:", soles1, "soles")
print()

print(pago.pagar_dolares(dolar2))
print("Equivale a:", soles2, "soles")
print()

print(pago.pagar_dolares(dolar3))
print("Equivale a:", soles3, "soles")
```

<https://www.onlinegdb.com/edit/Uc4lAEZul>

SALIDA ESPERADA

```
Pago realizado: 10 dolares
Equivale a: 37.0 soles

Pago realizado: 25 dolares
Equivale a: 92.5 soles

Pago realizado: 100 dolares
Equivale a: 370.0 soles
```

Decorator

También llamado: Decorador, Envoltorio, Wrapper.

Decorator Es un patrón estructural que permite agregar nuevas funcionalidades a un objeto sin modificar su clase original.

En otras palabras:

“Decorator envuelve un objeto para darle más características.”

[Aplicar un Decorator para añadir validación extra a funciones matemáticas.](#)

IMPLEMENTACIÓN EN PYTHON

```

# Decorator
def validar(func):

    def envoltura(a, b):

        # Validación extra
        if b == 0:
            print("Error: no se puede dividir entre cero")
            return

        if a < 0 or b < 0:
            print("Error: no se permiten numeros negativos")
            return

        # Ejecutar función original
        return func(a, b)

    return envoltura

# Función matemática
@validar
def dividir(a, b):
    return a / b

# Pruebas
print(dividir(10, 2))
print()

print(dividir(8, 0))
print()

print(dividir(-5, 2))
print()

print(dividir(20, 4))

```

<https://www.onlinegdb.com/edit/xZqv7o6wn>

SALIDA ESPERADA

```

5.0

Error: no se puede dividir entre cero
None

Error: no se permiten numeros negativos
None

5.0

```

Composite

También llamado: Objeto compuesto, Object tree.

El patrón de diseño Composite (Compuesto) es un patrón estructural que permite tratar objetos individuales y grupos de objetos de la misma manera.

Usar para crear una estructura jerárquica de carpetas y archivos Composite.

IMPLEMENTACIÓN EN C++

```
#include <iostream>
#include <vector>
using namespace std;

// COMPONENT
class Elemento {
public:
    virtual void mostrar() = 0;
};

// LEAF -> Archivo
class Archivo : public Elemento {
private:
    string nombre;

public:
    Archivo(string n) {
        nombre = n;
    }

    void mostrar() override {
        cout << "Archivo: " << nombre << endl;
    }
};

// COMPOSITE -> Carpeta
class Carpeta : public Elemento {
private:
    string nombre;
    vector<Elemento*> elementos;

public:
    Carpeta(string n) {
        nombre = n;
    }

    // Agregar archivos o carpetas
    void agregar(Elemento* elemento) {
        elementos.push_back(elemento);
    }

    void mostrar() override {
        cout << "Carpeta: " << nombre << endl;

        // Mostrar todo lo que contiene
        for (int i = 0; i < elementos.size(); i++) {
            elementos[i]->mostrar();
        }
    }
}
```

```

};

int main() {

    // Archivos
    Archivo* archivo1 = new Archivo("foto.png");
    Archivo* archivo2 = new Archivo("documento.pdf");
    Archivo* archivo3 = new Archivo("tarea.docx");

    // Carpetas
    Carpeta* carpeta1 = new Carpeta("Mis Archivos");
    Carpeta* carpeta2 = new Carpeta("Tareas");

    // Agregar archivos a carpeta2
    carpeta2->agregar(archivo3);

    // Agregar elementos a carpeta1
    carpeta1->agregar(archivo1);
    carpeta1->agregar(archivo2);
    carpeta1->agregar(carpeta2);

    // Mostrar estructura completa
    carpeta1->mostrar();

    return 0;
}

```

<https://www.onlinegdb.com/edit/4aVj2-63>

SALIDA ESPERADA

```

Carpeta: Mis Archivos
Archivo: foto.png
Archivo: documento.pdf
Carpeta: Tareas
Archivo: tarea.docx

```